Section 6.1 — LangGraph

Build Stateful, Production-Ready Agent Workflows

Author: Suresh D R | AI Product Developer & Technology Mentor

---

Table of Contents

#	Topic
6.1.1	What Is LangGraph
6.1.2	Why LangGraph Exists
6.1.3	The Five Core Components
6.1.4	State — The Heart of Everything
6.1.5	Nodes and Edges
6.1.6	How State Transfers Between Agents
6.1.7	No Agent-to-Agent Communication
6.1.8	Types of Workflows You Can Build
6.1.9	Real Use Case — Logistics Report Agent
6.1.10	Use Cases That Require LangGraph
6.1.11	When NOT to Use LangGraph
6.1.12	Section Summary

6.1.1 What Is LangGraph

LangGraph is a framework built on top of LangChain that lets you build **stateful, multi-step agent workflows** as a **graph**.

A graph is a set of **nodes** (individual steps) connected by **edges** (what happens next). The graph can branch, loop, pause for human approval, save its progress, and resume after a crash.

```
One sentence:
LangGraph turns your agent workflow into a controllable,
resumable, branchable structure — instead of a fragile linear loop.
```

Before LangGraph, an agent was a simple loop:

```
Think → Act → Observe → Think → Act → Observe → Answer
```

This works for simple tasks. The moment a task gets complex — needs decisions, retries, human approval, or must survive a crash — the simple loop breaks. LangGraph was built to fix this.

6.1.2 Why LangGraph Exists

Here are the four real problems LangGraph solves:

Problem 1 — No branching in a simple loop

```
What you need in production:
  "If the document is valid → process it"
  "If the document is invalid → ask user to resubmit"
  "If the refund is under Rs 1000 → auto-approve"
  "If the refund is above Rs 1000 → send to manager"

What a simple loop gives you:
  One path. No decisions.
  You have to write messy if/else code everywhere.
```

Real example: A Swiggy customer complaint agent. The message could be about a wrong order, a delayed delivery, or a billing issue. Each needs a completely different response flow. A simple loop cannot branch — it goes one path for everything. LangGraph routes each complaint type to the right specialist node

automatically.

LangGraph solution: conditional edges — the graph decides which node to run next based on what is in the state.

Problem 2 — No way to resume after a crash

```
Scenario: Agent processes a loan application — 20 steps.
          Crashes at step 14.
          Simple loop: starts from step 1 again.
          20 minutes of API calls — gone.

What you need: save progress, resume from step 14.
Simple loop: cannot do this.
```

Real example: An HDFC loan processing agent runs 20 steps — document check, credit score, fraud check, income verification, risk scoring, and so on. If the server crashes at step 16, restarting from step 1 means re-running API calls that already completed, potentially double-charging fees, and losing 25 minutes of work. The customer is still waiting.

LangGraph solution: checkpointing — state is saved automatically after every single node. Resume from step 16, not step 1.

Problem 3 — No human approval gates

```
Scenario: Agent drafts 500 emails to customers.
          You want to review before sending.

Simple loop: no built-in way to pause and wait.
You have to build this yourself.
```

Real example: A marketing agent at Myntra generates 10,000 personalised promotional emails. Before any email is sent, the marketing manager needs to review a sample and approve. With a simple loop there is no clean way to pause execution, show the drafts, wait for a button click, and then resume. You end up building this pause-resume system yourself — which is exactly what LangGraph provides out of the box.

LangGraph solution: `interrupt_before` — pause the graph at any node, wait for human input, resume.

Problem 4 — Hard to debug

```
Simple loop: one black box. If output is wrong you
             don't know which step caused it.

LangGraph: inspect the full state after every single node.
           You see exactly what every agent wrote at every step.
```

Real example: A research agent produces a competitor analysis report that has wrong data in section 3. With a simple loop you have no idea if the bad data came from the web search, the analysis step, or the writing step. You re-run the whole thing and hope. With LangGraph you inspect the state after each node — you can see that `analysis_data` was correct but `written_output` introduced an error — so the bug is in the writing node, not the research node. Problem found in 2 minutes instead of 2 hours.

6.1.3 The Five Core Components

LangGraph has five building blocks. Understand these five and you understand LangGraph completely.

---

1. State
A Python dictionary shared by all agents in the graph. Every agent reads from it and writes to it. This is the only way agents share information in LangGraph — through this shared state. There is no direct agent-to-agent communication.

2. Nodes
Individual steps in the workflow. Each node is a Python function that reads the state, does one piece of work (calls LLM, runs a tool, makes a decision), and writes updates back to the state.

3. Edges
Connections between nodes. Two types:

- **Direct edge** — always go from node A to node B
- **Conditional edge** — decide based on state which node to go to next

4. Graph
The container that holds all nodes and edges. You build the graph, compile it, and run it.

5. Checkpointer
Saves the full state after every node automatically. Enables crash recovery, human-in-the-loop, and resuming across sessions.

6.1.4 State — The Heart of Everything

> **Analogy:** Think of state like a shared whiteboard in the middle of a meeting room. Every person in the meeting (every node/agent) can read the whiteboard and write on it. Nobody passes notes privately to each other. Everything goes through the whiteboard. When one person writes something, the next person reads exactly what was written and adds their own contribution on top.

What State Looks Like

State is a Python TypedDict — a dictionary with defined fields. Here is a simple example for a delivery planning workflow:

```
from typing import TypedDict, List

class DeliveryState(TypedDict):
    request: str          # the original user request
    zone_facts: str       # what the research agent found
    delivery_plan: str    # what the planning agent created
    quality_score: int    # score from the evaluation agent (0-10)
    retry_count: int      # how many times planning has been attempted
    final_report: str     # the finished output
    status: str           # current status of the workflow
```

This one TypedDict is shared across ALL agents in the graph. Each agent reads the fields it needs and writes back only the fields it changed.

How State Updates at Each Step

When a node finishes, it returns ONLY the fields it changed. LangGraph merges this into the existing state automatically.

```
STEP 1: research_node runs

  Before:                          After:
  request: "Plan Bandra zone"      request: "Plan Bandra zone"  ← unchanged
  zone_facts: ""                   zone_facts: "Bandra: 35-40   ← UPDATED
  delivery_plan: ""                agents, peak 9-11AM"
  quality_score: 0                 delivery_plan: ""            ← unchanged
  status: "started"                status: "research_done"      ← UPDATED

STEP 2: planning_node runs (reads zone_facts, writes delivery_plan)

  Before:                          After:
  zone_facts: "Bandra: 35-40..."   zone_facts: "Bandra: 35-40..." ← unchanged
  delivery_plan: ""                delivery_plan: "Deploy 35      ← UPDATED
  retry_count: 0                   agents in 3 shifts..."
  status: "research_done"          retry_count: 1               ← UPDATED
                                   status: "planning_done"      ← UPDATED

STEP 3: evaluate_node runs (reads delivery_plan, writes quality_score)

  quality_score: 0  →  quality_score: 6   ← UPDATED
  status: "planning_done" → "evaluation_done" ← UPDATED
```

Each node only writes what it changed. The rest of the state stays exactly as it was.

6.1.5 Nodes and Edges

Nodes in Simple Terms

A node is a Python function. It receives the current state, does one specific job, and returns what changed.

```python
# Example: the research node
def research_node(state):
    # Read from state
```

```
    request = state["request"]

    # Do the work
    facts = llm.invoke("Give key delivery facts for: " + request)

    # Return ONLY what changed — not the whole state
    return {
        "zone_facts": facts.content,
        "status": "research_done"
    }
```

Each node does ONE thing. Nothing more.

Edges in Simple Terms

Direct edge — always go from A to B:

```
graph.add_edge("research_node", "planning_node")
# After research always comes planning. No condition.
```

Conditional edge — decide based on state:

```
def decide_next(state):
    if state["quality_score"] >= 7:
        return "output_node"    # good quality → deliver
    else:
        return "planning_node"  # poor quality → improve and retry

graph.add_conditional_edges("evaluate_node", decide_next, {
    "output_node":  "output_node",
    "planning_node": "planning_node"
})
```

6.1.6 How State Transfers Between Agents

This is the most important concept in LangGraph. Let us trace exactly how information moves from one agent to the next using a real example.

The Scenario

Arjun asks: *"Plan delivery operations for Bandra and Andheri zones in Mumbai"*

Three agents handle this: Research Agent → Planning Agent → Review Agent

---

Step-by-Step State Transfer

Starting State (Before Any Agent Runs)

```
STATE:
  request       : "Plan delivery for Bandra and Andheri zones"
  zone_facts    : ""       ← empty, nothing researched yet
  delivery_plan : ""       ← empty, nothing planned yet
  quality_score : 0        ← not evaluated yet
  final_report  : ""       ← no report yet
  status        : "started"
```

Research Agent Runs — Writes to State

The Research Agent reads `request` from state. It calls the LLM with the request. It writes its findings back to `zone_facts` in state.

```
RESEARCH AGENT:
  Reads from state: request = "Plan delivery for Bandra and Andheri zones"
  Calls LLM: "Give key facts for delivery in Bandra and Andheri..."
  LLM replies with zone facts
  Writes to state: zone_facts = "Bandra: 35-40 agents, peak 9-11AM.
                                 Andheri: 40-50 agents, WEH congestion."

STATE AFTER RESEARCH AGENT:
```

```
  request       : "Plan delivery for Bandra and Andheri zones"
  zone_facts    : "Bandra: 35-40 agents...Andheri: 40-50 agents..."  ← WRITTEN
  delivery_plan : ""
  quality_score : 0
  status        : "research_done"                                    ← WRITTEN
```

The Research Agent wrote two fields. Everything else is untouched.

Planning Agent Runs — Reads and Writes State

The Planning Agent does NOT call the Research Agent. It reads `zone_facts` from state — the same field the Research Agent wrote. It creates plans and writes them to `delivery_plan` in state.

```
PLANNING AGENT:
  Reads from state: zone_facts = "Bandra: 35-40 agents...Andheri: 40-50..."
  Calls LLM: "Based on these facts, create a delivery plan..."
  LLM creates the plan
  Writes to state: delivery_plan = "Bandra: Deploy 35 agents in 3 shifts,
                                    avoid WEH at peak hours.
                                    Andheri: 40 agents across 4 sub-zones."

STATE AFTER PLANNING AGENT:
  request       : "Plan delivery for Bandra and Andheri zones"
  zone_facts    : "Bandra: 35-40 agents...Andheri: 40-50 agents..."
  delivery_plan : "Bandra: Deploy 35 agents...Andheri: 40 agents..."  ← WRITTEN
  quality_score : 0
  status        : "planning_done"                                     ← WRITTEN
```

The Planning Agent never called the Research Agent. It just read the whiteboard.

Review Agent Runs — Reads and Writes State

The Review Agent reads `delivery_plan` from state. It evaluates quality and writes a score to `quality_score`.

```
REVIEW AGENT:
  Reads from state: delivery_plan = "Bandra: Deploy 35 agents..."
  Calls LLM: "Rate this plan from 1-10..."
  LLM replies: "8"
  Writes to state: quality_score = 8

STATE AFTER REVIEW AGENT:
  request       : "Plan delivery for Bandra and Andheri zones"
  zone_facts    : "Bandra: 35-40 agents...Andheri: 40-50 agents..."
  delivery_plan : "Bandra: Deploy 35 agents...Andheri: 40 agents..."
  quality_score : 8                                          ← WRITTEN
  status        : "review_done"                              ← WRITTEN
```

The Routing Decision — Conditional Edge Reads State

After the Review Agent, the conditional edge routing function reads `quality_score` from state and decides:

```
quality_score = 8
8 >= 7 → route to output_node → deliver the plan

If quality_score was 5:
5 < 7 → route back to planning_node → improve and retry
```

The state itself drives the routing. No external logic needed.

Final State (After All Agents)

```
STATE:
  request       : "Plan delivery for Bandra and Andheri zones"
  zone_facts    : "Bandra: 35-40 agents...Andheri: 40-50 agents..."
  delivery_plan : "Bandra: Deploy 35 agents...Andheri: 40 agents..."
  quality_score : 8
```

```
  final_report  : "DELIVERY PLAN — Bandra and Andheri\n..."
  status        : "complete"
```

Every field was written by a different agent. Every agent read only what it needed. Nobody called anybody. The state is the complete record of everything that happened.

6.1.7 No Agent-to-Agent Communication

> ⚠️ **Critical Rule: In LangGraph, agents do NOT talk to each other.**

This is the single most important thing to understand about LangGraph.

```
What people expect:
  Research Agent → sends result → Planning Agent → sends plan → Review Agent

How LangGraph actually works:
  Research Agent → writes to STATE
                          ↓
              Planning Agent reads from STATE
              Planning Agent → writes to STATE
                                    ↓
                        Review Agent reads from STATE
```

The graph controls the ORDER. State is the MESSENGER. Agents have no knowledge of each other.

Why This Is Better

```
Direct agent-to-agent:                State-based (LangGraph):
  Agent A must know Agent B             Agent A only knows the state fields
  Change Agent B → might break Agent A  Change Agent B → Agent A unaffected
  Crash at Agent B → Agent A fails      Crash anywhere → state is intact, resume
  Hard to inspect what was passed       Full state visible at every step
  Cannot add agent between A and B      Add a node, connect with edges — done
```

What To Use If You Need Agents Talking To Each Other

LangGraph is NOT the right choice for direct agent communication. Use:

```
AutoGen  → agents literally message each other in a chat
           Agent A writes → Agent B critiques → Agent A improves
           Best for: debate, iterative writing, back-and-forth

CrewAI   → structured task delegation between role-based agents
           Best for: team workflows with clear roles

LangGraph → workflow orchestration through shared state
            Best for: complex pipelines, branching, checkpointing
```

6.1.8 Types of Workflows You Can Build

Linear Pipeline — A → B → C → D → Done
Research → Analyse → Write → Format → Deliver. Each step depends on the previous.

Branching — A → decide → B or C
Customer message arrives → classify → route to delivery agent or billing agent or complaint agent. Different customer types go different paths.

Looping (Self-Improving) — A → B → C → back to A if quality poor
Write a plan → evaluate quality → if score < 7 loop back and improve → if score >= 7 deliver. The agent improves its own output automatically.

Multi-Agent Coordinator — One orchestrator, multiple specialist nodes
Orchestrator identifies 3 zones → Zone A agent → Zone B agent → Zone C agent (run one by one or setup parallel logic) → assembler combines results.

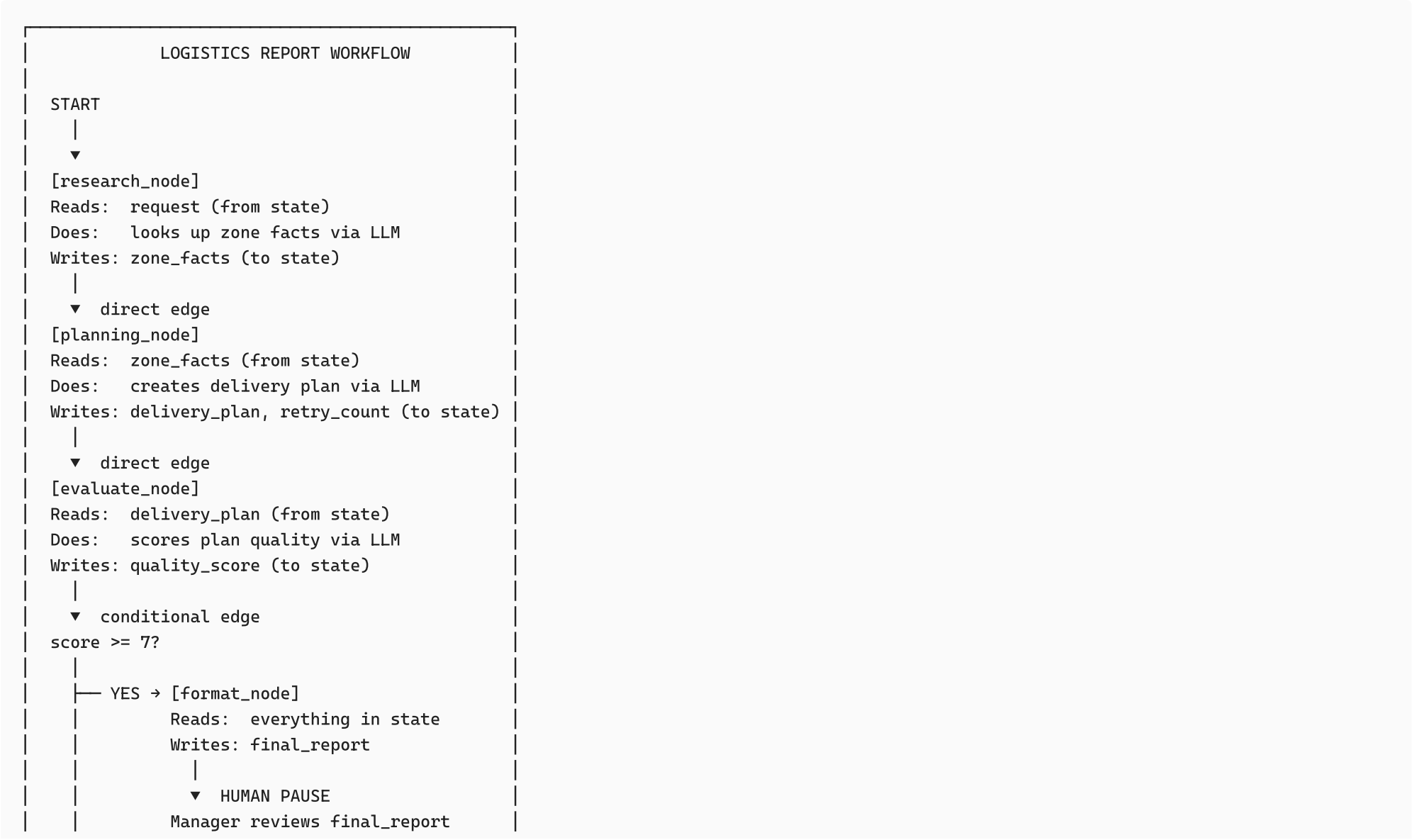
Human-in-the-Loop — Run → pause → human approves → continue
Agent drafts emails → graph pauses → human reviews → human approves → agent sends. The irreversible action only happens after human sign-off.

6.1.9 Real Use Case — Logistics Report Agent

The Scenario

Arjun manages a logistics company. Every Monday morning he needs a delivery planning report for 3 Mumbai zones — Bandra, Andheri, and Dadar. The report must be high quality (score >= 7) before it is delivered. And a human manager must review it before it is finalised.

The Workflow



```
|    |         Manager adds notes if needed      |
|    |           |                               |
|    |           ▼                               |
|    |          END                              |
|    |                                           |
|    └── NO  → [planning_node] ← loops back      |
|              improves the plan                 |
|              re-evaluates                      |
|              (max 3 attempts)                  |
└────────────────────────────────────────────────┘
```

How State Flows Through This Workflow — Step by Step

The state is a single shared dictionary. Every node reads from it and writes to it. Here is how it changes at every step.

Starting State — Nothing Has Run Yet

```
request       →  "Plan Bandra, Andheri, Dadar zones for Monday"   (set by user)
zone_facts    →  ""                                                (empty)
delivery_plan →  ""                                                (empty)
quality_score →  0                                                 (not evaluated)
retry_count   →  0                                                 (no attempts)
human_notes   →  ""                                                (empty)
final_report  →  ""                                                (empty)
status        →  "started"
```

After research_node

Reads from state: `request`
Writes to state: `zone_facts` , `status`

```
request       →  "Plan Bandra, Andheri, Dadar zones for Monday"   (unchanged)
zone_facts    →  "Bandra: 35-40 agents, peak 9-11AM and 5-8PM.   ← WRITTEN
```

```
                  Andheri: 40–50 agents, WEH congestion.
                  Dadar: 25–30 agents, busy all day."
delivery_plan →  ""                                          (unchanged)
quality_score →  0                                           (unchanged)
retry_count   →  0                                           (unchanged)
human_notes   →  ""                                          (unchanged)
final_report  →  ""                                          (unchanged)
status        →  "research_done"                             ← WRITTEN
```

After planning_node — Attempt 1

Reads from state: `zone_facts`
Writes to state: `delivery_plan` , `retry_count` , `status`

```
zone_facts    →  "Bandra: 35–40 agents..."                    (unchanged)
delivery_plan →  "Deploy 35 agents in Bandra across 3 shifts.     ← WRITTEN
                  Andheri: 40 agents, avoid WEH peak hours.
                  Dadar: 28 agents, staggered timings."
quality_score →  0                                           (unchanged)
retry_count   →  1                                           ← WRITTEN (was 0)
status        →  "planning_done"                             ← WRITTEN
```

After evaluate_node — Attempt 1

Reads from state: `delivery_plan`
Writes to state: `quality_score` , `status`

```
delivery_plan →  "Deploy 35 agents in Bandra..."             (unchanged)
quality_score →  5                                           ← WRITTEN
retry_count   →  1                                           (unchanged)
status        →  "evaluation_done"                           ← WRITTEN
```

> **Routing decision:** quality_score = 5, retry_count = 1
> Score is below 7 and retries are below 3 → **loop back to planning_node**

After planning_node — Attempt 2

Reads from state: `zone_facts`, `delivery_plan` (reads old plan to improve it)
Writes to state: `delivery_plan`, `retry_count`, `status`

```
delivery_plan →  "IMPROVED: Deploy 37 agents in Bandra with       ← WRITTEN
                  30-min peak hour buffer 9-11AM and 5-8PM.
                  Andheri: 42 agents in 4 sub-zones.
                  Dadar: 28 agents, 2 shifts with overlap."
quality_score →  5                                                 (unchanged — still old score)
retry_count   →  2                                                 ← WRITTEN (was 1)
status        →  "planning_done"                                   ← WRITTEN
```

After evaluate_node — Attempt 2

Reads from state: `delivery_plan` (the improved version now)
Writes to state: `quality_score`, `status`

```
delivery_plan →  "IMPROVED: Deploy 37 agents..."                   (unchanged)
quality_score →  8                                                 ← WRITTEN
retry_count   →  2                                                 (unchanged)
status        →  "evaluation_done"                                 ← WRITTEN
```

> **Routing decision:** quality_score = 8 → score is >= 7 → **proceed to format_node**

After format_node

Reads from state: `request`, `zone_facts`, `delivery_plan`, `quality_score`, `retry_count`
Writes to state: `final_report`, `status`

```
final_report  →  "MONDAY LOGISTICS REPORT                          ← WRITTEN
                  Zones: Bandra, Andheri, Dadar
                  Quality Score: 8/10 | Attempts: 2
                  Bandra: 37 agents with peak hour buffer...
                  Andheri: 42 agents across 4 sub-zones...
                  Dadar: 28 agents, 2 shifts with overlap..."
status        →  "awaiting_human_approval"                          ← WRITTEN
```

> **Graph pauses here** — waiting for manager to review

Human Review

Manager reads: `final_report`
Manager writes into state: `human_notes`, `status`

```
final_report  →  "MONDAY LOGISTICS REPORT..."                       (unchanged)
human_notes   →  "Approved. Increase Andheri by 5 agents          ← WRITTEN
                  this week due to Diwali deliveries."
status        →  "human_approved"                                   ← WRITTEN
```

> **Graph resumes**

Final State — Workflow Complete

```
request       →  "Plan Bandra, Andheri, Dadar zones for Monday"
zone_facts    →  "Bandra: 35-40 agents... Andheri: 40-50..."
delivery_plan →  "IMPROVED: Deploy 37 agents in Bandra..."
quality_score →  8
retry_count   →  2
human_notes   →  "Approved. Increase Andheri by 5 agents..."
```

```
final_report  →  "MONDAY LOGISTICS REPORT..."
status        →  "complete"
```

Every field was written by a different agent at a different step. No agent called any other agent. The state IS the communication channel. You can read exactly what happened just by reading the final state.

This is LangGraph in action — state flowing through agents, improving through a loop, pausing for human approval, all without any agent ever calling another.

6.1.10 Use Cases That REQUIRE LangGraph

These are two real-world agent scenarios where a simple loop breaks and LangGraph is the right tool. In both cases, agents communicate through shared state — there is no direct agent-to-agent communication.

Use Case 1 — Travel Planning Agent

Scenario: A user says: *"Plan a 5-day trip to Rajasthan for 2 people in December, budget Rs 80,000."*

This is not a simple one-shot question. The agent needs to research flights, hotels, activities, and local transport — calculate the total cost — and if the budget is exceeded, it must find cheaper alternatives and recalculate. Finally, the user must approve the itinerary before anything is booked.

Why a simple agent loop fails here:

- No way to branch — if budget is exceeded, the loop has no way to take a different path
- No quality gate — agent cannot loop back and try cheaper options
- No human pause — cannot wait for user approval before booking

The LangGraph workflow:

Node	Reads from State	Writes to State
understand_request	user_request	destination, dates, budget, preferences
research_flights	destination, dates	flight_options, flight_cost
research_hotels	destination, dates, budget	hotel_options, hotel_cost

Node	Reads from State	Writes to State
research_activities	destination, preferences	activity_list, activity_cost
calculate_total	flight_cost, hotel_cost, activity_cost	total_cost, budget_status
adjust_plan	hotel_options, flight_options, total_cost	adjusted_selections
create_itinerary	all researched data	day_by_day_plan, booking_links
send_summary	final itinerary	confirmation_sent

The routing logic:

After calculate_total — if `total_cost <= budget` → proceed to create_itinerary. If `total_cost > budget` → go to adjust_plan (find cheaper options) → recalculate → loop until within budget.

After create_itinerary — **human pause** — user reviews the itinerary, approves or requests changes. Graph resumes only after approval.

State carries throughout: destination, travel dates, budget limit, all flight and hotel options found, selected choices, total cost, adjustment history, final itinerary, user approval — all shared across every node without any node calling another.

Use Case 2 — Medical Report Analyser

Scenario: A patient uploads a blood test report PDF. The system extracts all values, flags which ones are abnormal, determines the severity level (normal / mild / critical), generates a patient-friendly summary, and routes the output based on severity. No medical output ever reaches the patient without a doctor reviewing it first.

Why a simple agent loop fails here:

- No branching — cannot route critical results to a doctor alert separately from mild results
- No human gate — sending medical advice to a patient without doctor review is dangerous
- No state — if the analysis crashes midway, the extracted values are lost and the whole PDF must be re-processed

The LangGraph workflow:

Node	Reads from State	Writes to State
extract_values	report_pdf_text	test_results (all values)
flag_abnormal	test_results	abnormal_values, severity_level
generate_advice	abnormal_values (mild cases)	patient_advice, followup_recommendation
alert_doctor	abnormal_values, severity_level (critical)	doctor_alert, urgency_flag
generate_summary	patient_advice or doctor_alert	patient_friendly_summary
send_to_patient	patient_friendly_summary, doctor_approval	delivery_status

The routing logic:

After flag_abnormal — three paths based on `severity_level` in state:

- `"normal"` → generate standard summary → doctor reviews → send to patient
- `"mild"` → generate advice → **human pause** (doctor reviews) → send to patient
- `"critical"` → alert doctor immediately → **human pause** (mandatory) → doctor edits if needed → send to patient

Why the human pause is mandatory here: the graph physically cannot proceed to send_to_patient without a doctor writing their approval into the state. The graph is paused and waiting. No code workaround is possible — the architecture enforces the safety gate.

State carries throughout: raw PDF text, all extracted test values, abnormal flags, severity classification, doctor's notes, patient-friendly summary, approval status — all in one shared object. You can inspect the complete state at any point and see exactly what was found and decided at every step.

What Both Use Cases Have in Common

Both examples share the same four characteristics that make LangGraph the right tool:

Multiple sequential steps where each depends on the previous output. A simple one-shot LLM call cannot handle this.

Decision branches that change the path based on what is found. Travel: budget exceeded → adjust. Medical: critical severity → immediate doctor alert.

At least one irreversible action that needs a human gate. Booking a flight. Sending medical advice to a patient. The graph pauses and genuinely waits — it cannot be bypassed.

Must survive a crash. If the travel planning workflow crashes after flights and hotels are researched but before the itinerary is created, checkpointing means it resumes from that point — not from the beginning. The research does not need to be redone.

In both cases, no agent calls any other agent. Every agent reads from state and writes to state. The graph controls the order. State is the only messenger.

6.1.11 When NOT to Use LangGraph

```
Simple question-answer agent:
  "What is the fuel cost from Pune to Mumbai?"
  Use plain AgentExecutor. LangGraph adds unnecessary complexity.

Learning agents for the first time:
  Start with Section 2's plain ReAct agent.
  Understand the loop before adding graph structure.

Quick prototype or demo:
  Speed matters more than robustness.
  A plain agent takes 10 minutes to build.
  A LangGraph workflow takes 45 minutes.
  For a demo, use the plain agent.

Agents that need to debate and critique each other:
  LangGraph agents communicate through state — not through messages.
  For debate and back-and-forth: use AutoGen.
  For role-based teams: use CrewAI.
```

6.1.12 Section Summary

LangGraph is not a replacement for your agent logic. It is the **structure** that holds your agent workflow together — making it branchable, resumable, inspectable, and production-ready.

The Five Core Components

Component	What It Is	What It Does
State	TypedDict dictionary	Holds all information flowing through the graph
Nodes	Python functions	Each does one piece of work, reads/writes state
Edges	Connections between nodes	Direct (always) or Conditional (decide based on state)
Graph	Container for nodes + edges	Compiled and executed
Checkpointer	State persistence layer	Saves after every node, enables resume

---

The Most Important Rules

Rule 1 — No direct agent-to-agent communication.
Agents communicate ONLY through shared state. Agent A writes to state. Agent B reads from state. The graph controls the order. Nobody calls anybody.

Rule 2 — Each node does one thing.
Read from state. Do one job. Write what changed to state. Nothing more.

Rule 3 — Conditional edges drive all decisions.
The routing function reads the state and returns which node to run next. All branching logic lives here.

Rule 4 — Start simple.
Do you need branching, looping, checkpointing, or human-in-the-loop? If yes — use LangGraph. If no — use plain AgentExecutor.

6.1.13 Tool Calling in LangGraph

So far the nodes called the LLM directly with `llm.invoke()`. In production, agents need tools — web search, calculators, databases, APIs. LangGraph handles tool calling cleanly by giving the LLM a list of tools, letting it decide which tool to call, running it, and feeding the result back through state.

How Tool Calling Works in LangGraph

The pattern has three parts:

1. Define your tools using the `@tool` decorator — same as in Section 2.

2. Bind tools to the LLM using `llm.bind_tools(tools)` — this tells the LLM what tools are available.

3. Add a tool execution node — a node that checks if the LLM requested a tool call, runs the tool, and writes the result back to state.

```
from langchain_openai import ChatOpenAI
from langchain.tools import tool
from langgraph.graph import StateGraph, END
from typing import TypedDict, List, Annotated
import operator

# Step 1: Define tools
@tool
def calculator(expression: str) -> str:
    "Evaluate a math expression like 23 * 4 or 180 / 12"
    try:
        return "Result: " + str(eval(expression))
    except Exception as e:
        return "Error: " + str(e)

@tool
def knowledge_base(query: str) -> str:
    "Look up facts about logistics, cities, fuel prices"
    facts = {
        "pune mumbai": "Distance Pune to Mumbai toll-free via SV Road: 180 km.",
        "fuel":        "Petrol price in India: Rs 95 per litre.",
        "bandra":      "Bandra zone Mumbai: 35-40 agents, peak 9-11AM and 5-8PM.",
        "andheri":     "Andheri zone Mumbai: 40-50 agents, heavy WEH traffic.",
    }
    for key, val in facts.items():
        if any(w in query.lower() for w in key.split()):
            return val
    return "Not found."
```

```
tools = [calculator, knowledge_base]

# Step 2: Bind tools to LLM
llm = ChatOpenAI(model="gpt-3.5-turbo", temperature=0)
llm_with_tools = llm.bind_tools(tools)   # LLM now knows about the tools
```

The Tool Node — Running the Tool and Writing to State

When the LLM decides to call a tool, it returns a message with `tool_calls` inside it. The tool node reads this, runs the right tool, and writes the result back to state.

```
from langchain_core.messages import ToolMessage

def run_tools_node(state):
    """
    Checks if the last LLM message contains tool calls.
    If yes — runs each tool and writes results to messages in state.
    """
    last_message = state["messages"][-1]
    tool_results = []

    for tool_call in last_message.tool_calls:
        # Find the right tool by name
        tool_map = {t.name: t for t in tools}
        tool_fn = tool_map[tool_call["name"]]

        # Run the tool
        result = tool_fn.invoke(tool_call["args"])

        # Create a ToolMessage with the result
        tool_results.append(
            ToolMessage(
                content=result,
                tool_call_id=tool_call["id"]
            )
        )
```

```
    return {"messages": tool_results}   # adds to messages list in state
```

The Routing Logic — Should We Run a Tool or Are We Done?

After the LLM responds, we check: did it ask to call a tool, or did it give a final answer?

```
def should_call_tool(state) -> str:
    last_message = state["messages"][-1]

    if hasattr(last_message, "tool_calls") and last_message.tool_calls:
        return "run_tools"   # LLM wants to call a tool → run it
    else:
        return "done"        # LLM gave a final answer → we are finished
```

Complete End-to-End Code — Delivery Cost Agent with Tools

This is a complete, runnable LangGraph agent that uses tools. It calculates the fuel cost for a delivery route by searching the knowledge base and using the calculator.

```
import warnings, logging, os
warnings.filterwarnings("ignore")
logging.disable(logging.CRITICAL)

from typing import TypedDict, List, Annotated
import operator

from langchain_openai import ChatOpenAI
from langchain.tools import tool
from langchain_core.messages import HumanMessage, AIMessage, ToolMessage, SystemMessage
from langgraph.graph import StateGraph, END

# ── STEP 1: Define State ──────────────────────────────────────
```

```python
class AgentState(TypedDict):
    # messages accumulates — each new message is added, not replaced
    messages: Annotated[List, operator.add]

# ── STEP 2: Define Tools ──────────────────────────────────────
@tool
def calculator(expression: str) -> str:
    "Evaluate a math expression. Example: 180 / 12 * 95"
    try:
        return "Result: " + str(round(eval(expression), 2))
    except Exception as e:
        return "Error: " + str(e)

@tool
def knowledge_base(query: str) -> str:
    "Look up facts about routes, fuel prices, delivery zones."
    facts = {
        "pune mumbai":  "Distance Pune to Mumbai on toll-free SV Road: 180 km.",
        "fuel":         "Current petrol price in India: Rs 95 per litre.",
        "bandra":       "Bandra zone: 35-40 delivery agents, peak hours 9-11AM and 5-8PM.",
        "andheri":      "Andheri zone: 40-50 agents, heavy traffic on Western Express Highway.",
        "mileage":      "Standard delivery van mileage: 12 km per litre.",
    }
    for key, val in facts.items():
        if any(w in query.lower() for w in key.split()):
            return val
    return "Fact not found. Try: pune mumbai, fuel, bandra, andheri, mileage."

tools = [calculator, knowledge_base]

# ── STEP 3: Create LLM with tools bound ──────────────────────
llm = ChatOpenAI(model="gpt-3.5-turbo", temperature=0)
llm_with_tools = llm.bind_tools(tools)

# ── STEP 4: Define Nodes ──────────────────────────────────────
def agent_node(state: AgentState) -> dict:
    """The agent thinks and decides what to do next."""
    print("Agent thinking...")
    response = llm_with_tools.invoke(state["messages"])
```

```python
    return {"messages": [response]}

def tool_node(state: AgentState) -> dict:
    """Runs the tool the agent requested."""
    last_message = state["messages"][-1]
    tool_map = {t.name: t for t in tools}
    results = []

    for tool_call in last_message.tool_calls:
        tool_name = tool_call["name"]
        tool_args = tool_call["args"]
        print(f"Running tool: {tool_name}({tool_args})")

        result = tool_map[tool_name].invoke(tool_args)
        print(f"Tool result : {result}")

        results.append(
            ToolMessage(
                content=result,
                tool_call_id=tool_call["id"]
            )
        )

    return {"messages": results}

# ── STEP 5: Routing Function ──────────────────────────────
def should_use_tool(state: AgentState) -> str:
    last_message = state["messages"][-1]
    if hasattr(last_message, "tool_calls") and last_message.tool_calls:
        return "tool_node"   # agent wants a tool
    return END               # agent is done

# ── STEP 6: Build the Graph ───────────────────────────────
graph = StateGraph(AgentState)

graph.add_node("agent_node", agent_node)
graph.add_node("tool_node",  tool_node)

graph.set_entry_point("agent_node")
```

```python
# After agent runs — check if it wants a tool or is done
graph.add_conditional_edges(
    "agent_node",
    should_use_tool,
    {
        "tool_node": "tool_node",
        END: END
    }
)

# After tool runs — always go back to agent (agent reads result and continues)
graph.add_edge("tool_node", "agent_node")

# Compile
app = graph.compile()

# ── STEP 7: Run the Agent ──────────────────────────────────
print("=" * 55)
print("QUESTION: Fuel cost for Pune to Mumbai delivery van?")
print("=" * 55)
print("")

initial_state = {
    "messages": [
        SystemMessage(content=(
            "You are a logistics assistant. "
            "Use the knowledge_base tool to find facts. "
            "Use the calculator tool to compute results. "
            "Show your working step by step."
        )),
        HumanMessage(content=(
            "What is the fuel cost for a delivery van travelling "
            "from Pune to Mumbai on the toll-free route? "
            "Look up the distance, the petrol price, and the van mileage. "
            "Then calculate the total fuel cost."
        ))
    ]
}
```

```python
result = app.invoke(initial_state)

print("")
print("=" * 55)
print("FINAL ANSWER:")
print(result["messages"][-1].content)
```

What Happens When You Run This

```
QUESTION: Fuel cost for Pune to Mumbai delivery van?

Agent thinking...
Running tool: knowledge_base({'query': 'pune mumbai distance toll free'})
Tool result : Distance Pune to Mumbai on toll-free SV Road: 180 km.

Agent thinking...
Running tool: knowledge_base({'query': 'fuel petrol price India'})
Tool result : Current petrol price in India: Rs 95 per litre.

Agent thinking...
Running tool: knowledge_base({'query': 'delivery van mileage'})
Tool result : Standard delivery van mileage: 12 km per litre.

Agent thinking...
Running tool: calculator({'expression': '180 / 12 * 95'})
Tool result : Result: 1425.0

Agent thinking...
(No more tool calls - agent writes final answer)

FINAL ANSWER:
Here is the fuel cost calculation for the Pune to Mumbai delivery:

- Distance (toll-free SV Road) : 180 km
- Van mileage                  : 12 km per litre
```

```
- Petrol price                : Rs 95 per litre
- Litres needed               : 180 ÷ 12 = 15 litres
- Total fuel cost             : 15 × Rs 95 = Rs 1,425
```

The Tool Calling Loop — How It Works

```
START
  |
  ▼
[agent_node]
  LLM reads messages in state
  LLM decides: do I need a tool?
  |
  ├── YES: tool_calls in response
  |          |
  |          ▼
  |       [tool_node]
  |         runs the tool
  |         writes ToolMessage to state
  |         |
  |         └──→ back to [agent_node]
  |                LLM reads tool result
  |                decides: do I need another tool?
  |                (loop continues until no more tools needed)
  |
  └── NO: final answer in response
            |
            ▼
           END
```

The agent loops between `agent_node` and `tool_node` until it has all the information it needs. Then it writes its final answer and the graph ends. The number of tool calls is not fixed — the agent decides how many it needs.
